

```
zip_path = "/content/drive/MyDrive/shoe.zip"
extract_path = "/content/edges2shoes"
```

```
import zipfile

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall("/content/")

print("Dataset extracted successfully")
```

Dataset extracted successfully

```
import os

print("Train images:", len(os.listdir("/content/train")))
print("Validation images:", len(os.listdir("/content/val")))
```

Train images: 49825
Validation images: 200

```
!pip install torch torchvision tqdm matplotlib
```

```
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/py
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dis
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/d
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dis
Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/loca
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/li
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/li
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/pyt
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/p
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/py
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/l
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/
```

```
Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.12/dist
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.1
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/d
```

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.transforms as transforms
from torchvision.utils import save_image
from torch.utils.data import Dataset, DataLoader

import matplotlib.pyplot as plt
from PIL import Image
import os
from tqdm import tqdm
import os
```

```
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

IMG_SIZE = 256
BATCH_SIZE = 8
EPOCHS = 50

LR = 0.0002
BETA1 = 0.5
L1_LAMBDA = 100
```

```
BASE_DIR = "/content/drive/MyDrive/pix2pix_lab"

CHECKPOINT_DIR = f"{BASE_DIR}/checkpoints"
OUTPUT_DIR = f"{BASE_DIR}/outputs"

os.makedirs(CHECKPOINT_DIR, exist_ok=True)
os.makedirs(OUTPUT_DIR, exist_ok=True)
```

Start coding or [generate](#) with AI.

```
class Pix2PixDataset(Dataset):

    def __init__(self, root_dir):

        self.root_dir = root_dir
        self.images = os.listdir(root_dir)

        self.transform = transforms.Compose([
            transforms.Resize((256,512)),
```

```

        transforms.ToTensor(),
        transforms.Normalize((0.5,0.5,0.5),(0.5,0.5,0.5))
    ])

def __len__(self):
    return len(self.images)

def __getitem__(self, index):

    img_path = os.path.join(self.root_dir, self.images[index])
    image = Image.open(img_path).convert("RGB")

    image = self.transform(image)

    input_image = image[:, :, :256]
    target_image = image[:, :, 256:]

    return input_image, target_image

```

```

train_dataset = Pix2PixDataset("/content/train")
val_dataset = Pix2PixDataset("/content/val")

train_loader = DataLoader(train_dataset,
                           batch_size=BATCH_SIZE,
                           shuffle=True)

val_loader = DataLoader(val_dataset,
                        batch_size=1,
                        shuffle=False)

```

```

train_dataset = Pix2PixDataset("/content/train")
val_dataset = Pix2PixDataset("/content/val")

train_loader = DataLoader(train_dataset,
                           batch_size=BATCH_SIZE,
                           shuffle=True)

val_loader = DataLoader(val_dataset,
                        batch_size=1,
                        shuffle=False)

```

```

class Generator(nn.Module):

    def __init__(self):
        super().__init__()

        def down(in_c,out_c):
            return nn.Sequential(
                nn.Conv2d(in_c,out_c,4,2,1),
                nn.BatchNorm2d(out_c),
                nn.LeakyReLU(0.2)
            )

```

```

def up(in_c,out_c):
    return nn.Sequential(
        nn.ConvTranspose2d(in_c,out_c,4,2,1),
        nn.BatchNorm2d(out_c),
        nn.ReLU()
    )

self.d1 = down(3,64)
self.d2 = down(64,128)
self.d3 = down(128,256)
self.d4 = down(256,512)

self.u1 = up(512,256)
self.u2 = up(512,128)
self.u3 = up(256,64)

self.final = nn.ConvTranspose2d(128,3,4,2,1)

def forward(self,x):

    d1 = self.d1(x)
    d2 = self.d2(d1)
    d3 = self.d3(d2)
    d4 = self.d4(d3)

    u1 = self.u1(d4)
    u1 = torch.cat([u1,d3],1)

    u2 = self.u2(u1)
    u2 = torch.cat([u2,d2],1)

    u3 = self.u3(u2)
    u3 = torch.cat([u3,d1],1)

    return torch.tanh(self.final(u3))

```

```

class Discriminator(nn.Module):

    def __init__(self):
        super().__init__()

        def block(in_c,out_c,stride):
            return nn.Sequential(
                nn.Conv2d(in_c,out_c,4,stride,1),
                nn.BatchNorm2d(out_c),
                nn.LeakyReLU(0.2)
            )

        self.model = nn.Sequential(
            block(6,64,2),
            block(64,128,2),
            block(128,256,2),
            nn.Conv2d(256,1,4,1,1)
        )

```

```
def forward(self,x,y):  
    x = torch.cat([x,y],1)  
    return self.model(x)
```

```
gen = Generator().to(DEVICE)  
disc = Discriminator().to(DEVICE)
```

```
BCE = nn.BCEWithLogitsLoss()  
L1 = nn.L1Loss()  
  
opt_gen = optim.Adam(gen.parameters(), lr=LR, betas=(BETA1,0.999))  
opt_disc = optim.Adam(disc.parameters(), lr=LR, betas=(BETA1,0.999))
```

```
start_epoch = 0  
  
gen_ckpt = f"{CHECKPOINT_DIR}/gen_latest.pth"  
disc_ckpt = f"{CHECKPOINT_DIR}/disc_latest.pth"  
  
if os.path.exists(gen_ckpt):  
  
    print("Loading checkpoint...")  
  
    gen_checkpoint = torch.load(gen_ckpt)  
    disc_checkpoint = torch.load(disc_ckpt)  
  
    gen.load_state_dict(gen_checkpoint["model"])  
    disc.load_state_dict(disc_checkpoint["model"])  
  
    opt_gen.load_state_dict(gen_checkpoint["optimizer"])  
    opt_disc.load_state_dict(disc_checkpoint["optimizer"])  
  
    start_epoch = gen_checkpoint["epoch"] + 1  
  
    print("Resuming from epoch:", start_epoch)
```

```
Loading checkpoint...  
Resuming from epoch: 20
```

```
for epoch in range(start_epoch, EPOCHS):  
  
    print(f"\nStarting Epoch {epoch}/{EPOCHS}")  
  
    loop = tqdm(train_loader, desc=f"Epoch {epoch}")  
  
    for idx,(x,y) in enumerate(loop):  
  
        x = x.to(DEVICE)  
        y = y.to(DEVICE)  
  
        fake = gen(x)
```

```

# Train Discriminator
real_pred = disc(x,y)
fake_pred = disc(x,fake.detach())

loss_real = BCE(real_pred, torch.ones_like(real_pred))
loss_fake = BCE(fake_pred, torch.zeros_like(fake_pred))

D_loss = (loss_real + loss_fake)/2

opt_disc.zero_grad()
D_loss.backward()
opt_disc.step()

# Train Generator
fake_pred = disc(x,fake)

adv_loss = BCE(fake_pred, torch.ones_like(fake_pred))
l1_loss = L1(fake,y) * L1_LAMBDA

G_loss = adv_loss + l1_loss

opt_gen.zero_grad()
G_loss.backward()
opt_gen.step()

loop.set_postfix(
    epoch=epoch,
    G_loss=G_loss.item(),
    D_loss=D_loss.item()
)

print(f"Finished Epoch {epoch}")

# Save checkpoint
torch.save({
    "epoch": epoch,
    "model": gen.state_dict(),
    "optimizer": opt_gen.state_dict()
}, f"{CHECKPOINT_DIR}/gen_latest.pth")

torch.save({
    "epoch": epoch,
    "model": disc.state_dict(),
    "optimizer": opt_disc.state_dict()
}, f"{CHECKPOINT_DIR}/disc_latest.pth")

```

```

gen.eval()

epoch_folder = f"{OUTPUT_DIR}/epoch_{epoch}"
os.makedirs(epoch_folder, exist_ok=True)

for idx,(x,y) in enumerate(val_loader):

    x = x.to(DEVICE)

```

```

with torch.no_grad():
    fake = gen(x)

    save_image(fake*0.5+0.5,
               f"{epoch_folder}/output_{idx}.png")

if idx == 20:
    break

```

```

-----
NameError                                Traceback (most recent call last)
/tmp/ipykernel_672/2415365032.py in <cell line: 0>()
      1 gen.eval()
      2
----> 3 epoch_folder = f"{OUTPUT_DIR}/epoch_{epoch}"
      4 os.makedirs(epoch_folder, exist_ok=True)
      5

NameError: name 'epoch' is not defined

```

Next steps: [Explain error](#)

```

import torch

best_path = "/content/drive/MyDrive/pix2pix_lab/checkpoints/best_generator.p

gen.load_state_dict(torch.load(best_path, map_location=DEVICE))
gen.to(DEVICE)
gen.eval()

print("Loaded best generator checkpoint")

```

Loaded best generator checkpoint

```

import os

OUTPUT_DIR = "/content/drive/MyDrive/pix2pix_lab/final_outputs"
os.makedirs(OUTPUT_DIR, exist_ok=True)

```

```

import matplotlib.pyplot as plt
from torchvision.utils import save_image

def denorm(x):
    return (x + 1) / 2

for idx, (x, y) in enumerate(val_loader):

    x = x.to(DEVICE)

    with torch.no_grad():
        fake = gen(x)

```

```
inp = denorm(x[0]).permute(1,2,0).cpu()
real = denorm(y[0]).permute(1,2,0)
gen_img = denorm(fake[0]).permute(1,2,0).cpu()

# show images
fig, ax = plt.subplots(1,3, figsize=(12,4))

ax[0].imshow(inp)
ax[0].set_title("Input Edge")

ax[1].imshow(real)
ax[1].set_title("Ground Truth")

ax[2].imshow(gen_img)
ax[2].set_title("Generated")

for a in ax:
    a.axis("off")

plt.show()

# save generated image
save_image(gen_img.permute(2,0,1),
            f"{OUTPUT_DIR}/generated_{idx}.png")

if idx == 10:
    break
```



Input Edge



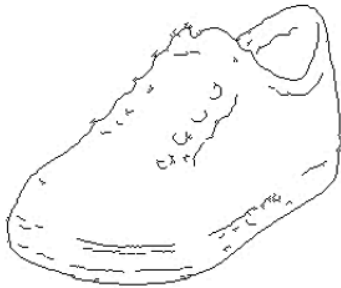
Ground Truth



Generated



Input Edge



Ground Truth



Generated



Input Edge



Ground Truth



Generated



Input Edge



Ground Truth



Generated



Input Edge



Ground Truth



Generated



Input Edge

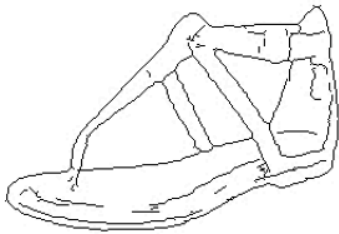


Ground Truth



Generated





Input Edge



Ground Truth



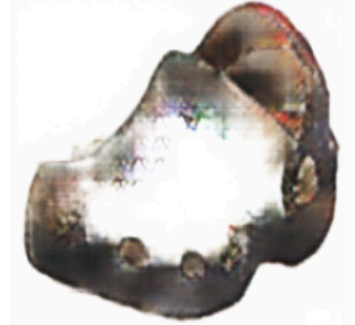
Generated



Input Edge



Ground Truth



Generated



Input Edge



Ground Truth



Generated



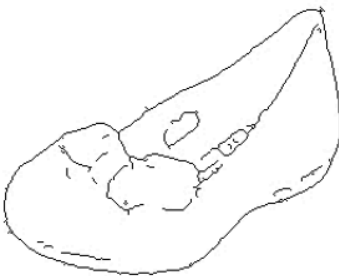
Input Edge



Ground Truth



Generated



Input Edge



Ground Truth



Generated



